KubeCon | CloudNativeCon
North America 2026

CLOUD NATIVE PLATFORM ENGINEERING COMMUNITY

PLATFORM ENGINEERING TCG · LIGHTNING TALKS

Closing the Gap Between Ready and Usable in a DBaaS Platform



KubeCon + CloudNativeCon NA 2026 · Kiosk P-16B, Hall 5

George Kechagias, Senior Software Engineer, Percona

PERCONA

TWO MINUTES AFTER THE REQUEST

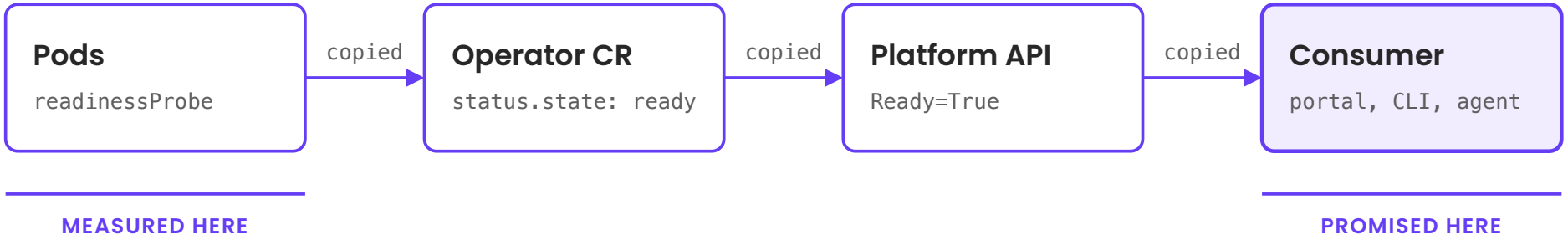
Ready. And the app can't connect.

```
$ kubectl get pg orders -n team-a
NAME     ENDPOINT                        STATUS   POSTGRES   PGBOUNCER   AGE
orders   orders-pgbouncer.team-a.svc     ready    3          1           2m
```

- The operator says `ready`. Whatever your platform puts on top copies it
- The engineer was promised they would never have to look underneath it

PERCONA®

Ready gets copied up the stack



What `ready` measures in Percona's PostgreSQL operator

`getState`, trimmed

```
if status.PGBouncer.Ready != status.PGBouncer.Size {
    return v2.AppStateInit
}
if status.Postgres.Ready != status.Postgres.Size {
    return v2.AppStateInit
}
return v2.AppStateReady
```

percona/controller/pgcluster/status.go · 52d3dc7

What makes those pods Ready

- Postgres: Patroni's `/readiness`
- PgBouncer: a `// TODO`, no readiness probe
- Nothing logs in as *your* app, so a wrong password, broken pooler auth or an untrusted cert all pass

PERCONA

Four things the app needs that `ready` never checks

The app needs	Why it can lag behind `ready`	What the app sees
Its role and password, in Postgres	The Secret is written first. Creating the role waits for a writable primary; until then the step returns `nil`	`password authentication failed`
PgBouncer to accept that role	PgBouncer checks passwords with `pgbouncer.get_auth`, which the operator installs the same way	Logins through the pooler fail
A server cert its CA trusts	With cert-manager, PgBouncer serves an internal cert until the real one is issued. `TLSSecretsReady` stays `True`	`verify-full` fails, then fixes itself
A network path from its namespace	NetworkPolicy, mesh and egress rules are platform-owned. No operator can see them	Connection timeout

Ask the same four questions of every engine you offer.

Your API has non-human consumers now

- Portals, CLIs, and agent tools all call the same API and read the same field
- An agent waits for Ready, reads the Secret, deploys. Nobody watches the gap
- The failure looks like **its** mistake, so it changes something: rotates the password, recreates the database, or turns off certificate verification
- A rotated password reopens the same gap: the Secret changes first, the SQL follows
- When the gap closes, those changes are still in place

Close the gap in the platform layer

Before testing, wait for its inputs

- `observedGeneration` is current, so the operator has at least seen the latest spec
- `status.host` is set, so there is something to dial
- The cert-manager Certificate is Ready, so a TLS failure means something

Then connect the way the app will

- Run it as a Job in the app's namespace, so network policy applies
- Read credentials from the Secret the app mounts, not the operator's original
- Connect with its `pgbouncer-uri` and `sslmode=verify-full`, then run `SELECT 1`
- Set Ready only when that passes

Operators can do part of this. A condition that logs in as each user through the pooler covers password, pooler and cert. The network path stays with the platform.

A status an agent can act on

```
status:
  phase: Initializing
  message: waiting for role orders_app in Postgres
  conditions:
  - type: Ready
    status: "False"
    reason: UsersPending
```

- `message` names the step the platform is still waiting for
- Split **wait** from **fix**: a pending reason means poll again, a blocking reason means stop and report
- Each row of the table gets its own reason, the network one included

Measure time to first query, not time to Ready

Time to Ready

What the platform reports today. Every provisioning dashboard has it.

Time to first query

What the engineer gets. The consumer-side check records it for free.

Takeaways

1. Read the function that writes your operator's ready, before you copy it up
2. Gate your own Ready on a real connection: from the consumer's namespace, with the consumer's Secret
3. Track time to first query next to time to Ready, and give every not-ready a reason
4. Tell agents what Ready means in your API: test the connection before deploying, wait on a pending reason, and never rotate credentials or relax `sslmode` to fix a connection error

Thank you

DOCUMENTATION

Percona Operators

MySQL, PostgreSQL and MongoDB on Kubernetes

docs.percona.com/percona-operators

FIND ME

George Kechagias

Senior Software Engineer, Percona

linkedin.com/in/george-kechagias-86118b92

PERCONA